



Centro Universitário Católica do Tocantins

Centro Universitário Católica do Tocantins

Engenharia de Software

Práticas Profissionais em Big Data

RELATÓRIO DE ATIVIDADE PRÁTICA
ORQUESTRAÇÃO DE PIPELINES DE ETL COM FERRAMENTAS OPEN
SOURCE

Apache Airflow aplicado ao Cenário NogTech

Profº Dr. Rogério Nogueira

Esther Frank

Palmas – TO, 09 de junho de 2026

SUMÁRIO

1. INTRODUÇÃO	4
2. DESENVOLVIMENTO	4
2.1. Contexto e Objetivo.....	4
2.2. Justificativa da Ferramenta Escolhida.....	5
2.2.1. Apache Airflow versus Apache NiFi	5
2.2.2. Apache Airflow versus Luigi	5
2.2.3. Síntese da Decisão	6
2.3. Arquitetura do Sistema.....	6
3. METODOLOGIA	7
3.1. Etapa de Extração (Extract)	8
3.1.1. Extração de Transações Financeiras (CSV)	8
3.1.2. Extração de Dados de Engajamento (JSON).....	8
3.2. Etapa de Transformação (Transform)	8
3.2.1. Padronização de CPF	9
3.2.2. Normalização de Datas.....	9
3.2.3. Junção das Fontes (LEFT JOIN)	9
3.2.4. Enriquecimento de Localização via BrasilAPI	9
3.2.5. Detecção de Feriados via BrasilAPI.....	10
3.2.6. Anonimização em conformidade com a LGPD	10
3.3. Etapa de Carga (Load).....	11
3.4. Infraestrutura e Containerização	11
3.4.1. Idempotência	11
3.4.2. Resiliência e Tratamento de Falhas.....	12

4. DIFICULDADES ENCONTRADAS	12
5. RESULTADOS.....	14
5.1. Resultados Esperados	14
5.2. Resultados Obtidos.....	14
5.2.1. Pipeline funcional e observável.....	14
5.2.2. Dados corretamente transformados e carregados.....	15
5.2.3. Idempotência verificada	15
5.2.4. Cache e resiliência funcionando corretamente	15
5.2.5. Pontos de melhoria identificados	15
6. CONCLUSÃO	16

1. INTRODUÇÃO

O presente relatório descreve o desenvolvimento de uma atividade prática da disciplina de Big Data, cujo objetivo central foi projetar, implementar e containerizar um pipeline de ETL (Extract, Transform, Load) utilizando uma ferramenta de orquestração open source. O cenário proposto envolveu a empresa fictícia NogTech, uma plataforma de cursos de tecnologia que necessitava consolidar, diariamente, um relatório cruzando transações financeiras com métricas de engajamento dos alunos.

A escolha da tecnologia de orquestração recaiu sobre o Apache Airflow, plataforma amplamente adotada no mercado de Engenharia de Dados. O pipeline desenvolvido integra arquivos estáticos locais (CSV e JSON) com dados dinâmicos consumidos em tempo real da BrasilAPI, aplicando transformações de negócio, enriquecimento geográfico, análise de calendário e anonimização de dados pessoais em conformidade com a Lei Geral de Proteção de Dados (LGPD).

Este documento está estruturado de forma a apresentar o contexto e os objetivos da atividade, a justificativa técnica da ferramenta escolhida, a metodologia aplicada em cada etapa do pipeline, as dificuldades encontradas durante o desenvolvimento, os resultados esperados e obtidos, e as considerações finais sobre o aprendizado proporcionado pela atividade.

2. DESENVOLVIMENTO

2.1. Contexto e Objetivo

A NogTech, plataforma fictícia de cursos de tecnologia, demandava um sistema capaz de consolidar diariamente informações provenientes de duas fontes distintas: o ERP financeiro, que exporta um arquivo CSV com dados de transações dos alunos, e a plataforma de vídeo, que exporta um arquivo JSON com métricas de engajamento mensais. O objetivo final era fornecer à diretoria uma visão unificada respondendo às seguintes perguntas de negócio:

- Quem está pagando pelos cursos e em qual plano?

- De onde vêm geograficamente esses alunos (cidade, estado, bairro)?
- As vendas ocorrem em datas comuns ou em feriados nacionais?
- Os alunos que pagam estão de fato consumindo a plataforma?

Para atingir esse objetivo, o pipeline deveria ser tolerante a falhas, garantir idempotência (possibilidade de reexecução sem duplicar dados) e aplicar anonimização dos dados pessoais conforme exigido pela LGPD. Todo o ecossistema deveria ser containerizado via Docker Compose, eliminando dependências do ambiente local de cada desenvolvedor.

2.2. Justificativa da Ferramenta Escolhida

A atividade oferecia três alternativas de orquestração: Apache Airflow, Apache NiFi e Luigi. A escolha pelo Apache Airflow foi fundamentada em critérios técnicos objetivos, discutidos a seguir em comparação com as demais opções.

2.2.1. Apache Airflow versus Apache NiFi

O Apache NiFi é uma ferramenta orientada a fluxo de dados em tempo real (streaming), com interface visual baseada em arrastar e soltar Processors numa tela chamada Canvas. Sua força reside em cenários de ingestão contínua de dados — IoT, mensageria, eventos em tempo real. Para o cenário da NogTech, contudo, o processamento é feito em lote (batch) uma vez ao dia, o que torna o NiFi uma solução superdimensionada.

Além disso, a configuração de Processors no NiFi é feita pela interface gráfica, o que dificulta o versionamento do fluxo em repositórios Git, a revisão de código em equipe e a automação de testes unitários da lógica de negócio. O Airflow, por outro lado, é configurado inteiramente em Python — cada DAG é um arquivo .py rastreável, revisável e testável como qualquer outro código-fonte.

2.2.2. Apache Airflow versus Luigi

O Luigi, desenvolvido pelo Spotify, é um framework Python orientado a alvos (targets): cada tarefa define um output() e o scheduler verifica se esse output já existe antes de reexecutar a tarefa. É uma abordagem elegante para pipelines simples. No

entanto, o Luigi não dispõe de uma interface web robusta — o Luigi Central Scheduler oferece uma UI básica, sem histórico detalhado de execuções, sem visualização de logs por tarefa e sem suporte nativo a agendamento por expressões cron sem configuração adicional.

O Airflow possui uma interface web completa que exibe o grafo de execução (DAG), o histórico de todas as execuções passadas, os logs individuais de cada tarefa, os tempos de duração e os dados trocados entre tarefas via XCom. Essa observabilidade é um requisito explícito da atividade e foi decisiva na escolha.

2.2.3. Síntese da Decisão

A tabela a seguir resume a comparação entre as três ferramentas nos critérios mais relevantes para o cenário NogTech:

Critério	Airflow	NiFi	Luigi
Paradigma	Batch / código Python	Streaming / visual	Batch / targets
Versionamento Git	Nativo (.py)	Difícil (XML/JSON)	Nativo (.py)
Interface web	Completa e detalhada	Completa, complexa	Básica
Agendamento cron	Nativo	Nativo	Manual / externo
Observabilidade	Alta	Alta	Baixa
Ecosistema Python	Total	Limitado	Total
Idempotência nativa	Re-run de DAG	Back-pressure	output() target
Curva de aprendizado	Moderada	Alta	Baixa

Fonte: elaboração própria (2026).

Diante da análise, o Apache Airflow apresentou o melhor equilíbrio entre expressividade em Python, observabilidade operacional e facilidade de agendamento para o processamento diário em batch exigido pelo cenário da NogTech.

2.3. Arquitetura do Sistema

O ecossistema é composto por cinco serviços Docker orquestrados via Docker Compose, todos conectados à mesma rede interna denominada nogtech-network:

postgres-airflow: Banco PostgreSQL 15 que armazena os metadados internos do Airflow: usuários, histórico de execuções, XComs e conexões. Acessível apenas dentro da rede Docker.

postgres-nogtech: Data Warehouse da NogTech. Banco PostgreSQL 15 exposto na porta 5433, destino final dos dados processados pelo pipeline. Contém a tabela fato_vendas.

airflow-init: Serviço de inicialização que executa as migrações do banco de metadados e cria o usuário administrador do Airflow. Executa uma única vez e encerra.

airflow-webserver: Interface web do Airflow, disponível na porta 8080. Permite monitorar o grafo de execução, o histórico, os logs e disparar execuções manualmente.

airflow-scheduler: Componente responsável por verificar os agendamentos definidos nas DAGs e disparar as execuções nas datas e horários configurados.

A inicialização dos serviços respeita uma cadeia de dependências explícita: o banco postgres-airflow deve estar saudável (healthcheck via pg_isready) antes de o airflow-init executar; e o airflow-init deve concluir com sucesso antes de o webserver e o scheduler subirem. Essa ordenação evita erros de conexão durante a inicialização do ecossistema.

Os dois bancos PostgreSQL possuem volumes Docker nomeados para persistência dos dados além do ciclo de vida dos containers. As pastas dags/, scripts/ e data/ são montadas diretamente do repositório local, permitindo que alterações no código sejam refletidas sem necessidade de rebuild da imagem Docker.

3. METODOLOGIA

O pipeline foi implementado seguindo o padrão ETL clássico em três fases sequenciais, orquestradas pelo Apache Airflow através de um DAG (Directed Acyclic Graph) denominado nogtech-pipeline. O DAG é agendado para execução diária às 06h00 por meio da expressão cron 0 6 * * *. O grafo de dependências entre as tarefas é

o seguinte: as duas tarefas de extração executam em paralelo; a transformação aguarda a conclusão de ambas; a carga aguarda a conclusão da transformação.

3.1. Etapa de Extração (Extract)

A extração é dividida em duas tarefas independentes, implementadas com PythonOperator, que executam em paralelo aproveitando a ausência de dependência entre si:

3.1.1. Extração de Transações Financeiras (CSV)

O arquivo `transacoes_nogtech.csv` é exportado pelo ERP financeiro com duas características que exigem configuração explícita na leitura: encoding latin-1 (ISO-8859-1), padrão em sistemas legados brasileiros para representação de caracteres acentuados; e delimitador ponto-e-vírgula (;), convenção adotada em exportações de planilhas no Brasil, onde a vírgula é reservada para casas decimais.

A leitura é feita com o módulo `csv.DictReader` da biblioteca padrão do Python, que mapeia cada linha do arquivo em um dicionário cujas chaves são os cabeçalhos das colunas. Após a leitura, os dados são empurrados para o mecanismo de comunicação entre tarefas do Airflow denominado XCom (Cross-Communication), via `xcom_push`, ficando disponíveis para a tarefa de transformação.

3.1.2. Extração de Dados de Engajamento (JSON)

O arquivo `engajamento_alunos.json` é exportado pela plataforma de vídeo em UTF-8, o encoding padrão moderno. Cada registro do arquivo contém métricas mensais de consumo por aluno: horas de conteúdo assistidas, quantidade de tickets de suporte abertos e pontuação NPS (Net Promoter Score). A leitura é feita com o módulo `json` da biblioteca padrão, e os dados são igualmente disponibilizados via XCom para a etapa seguinte.

3.2. Etapa de Transformação (Transform)

A transformação é a etapa de maior complexidade do pipeline, aplicando seis operações distintas sobre os dados extraídos. Todas as regras foram implementadas como obrigatórias, conforme especificado no enunciado da atividade.

3.2.1. Padronização de CPF

Os CPFs chegam nas fontes de dados em dois formatos distintos: com máscara (ex.: 123.456.789-00) e somente com dígitos (ex.: 12345678900). A função `padronizar_cpf()` utiliza expressão regular para remover todos os caracteres não numéricos e reconstrói o CPF no formato com máscara, validando que o resultado contém exatamente 11 dígitos. Registros com CPFs inválidos são descartados individualmente e registrados em log, sem interromper o processamento dos demais.

3.2.2. Normalização de Datas

As datas de transação aparecem em dois formatos: o formato brasileiro (DD/MM/AAAA) e o formato ISO 8601 (AAAA-MM-DD). A função `normalizar_data()` tenta parsear a data em cada formato sequencialmente utilizando `datetime.strptime()` e sempre retorna no padrão ISO 8601, que é o formato nativo do PostgreSQL para o tipo DATE. Registros com datas não reconhecidas são descartados.

3.2.3. Junção das Fontes (LEFT JOIN)

As transações são cruzadas com os dados de engajamento utilizando uma chave composta por dois campos: `cpf_aluno` (normalizado) e `mes_referencia` (os sete primeiros caracteres da data no formato AAAA-MM). Antes do loop principal de processamento, os registros de engajamento são indexados em um dicionário Python com essa chave composta, garantindo lookup em tempo $O(1)$ por transação. Quando não há engajamento correspondente para o mês da transação, os campos de engajamento recebem o valor `None` — comportamento equivalente a um LEFT JOIN em SQL.

3.2.4. Enriquecimento de Localização via BrasilAPI

Para cada CEP de cobrança presente nas transações, o pipeline consulta o endpoint público da BrasilAPI (<https://brasilapi.com.br/api/cep/v2/{CEP}>), que retorna

dados como cidade, estado e bairro. Esses três campos são adicionados ao registro final. Para evitar consultas repetidas à API para o mesmo CEP — o que desperdiçaria tempo de execução e sobrecarregaria o serviço público — os resultados são armazenados em um cache local no formato JSON em `/opt/airflow/cache/cep_cache.json`. O cache é carregado no início da transformação e salvo ao término, garantindo que CEPs já consultados em execuções anteriores não precisem ser consultados novamente.

3.2.5. Detecção de Feriados via BrasilAPI

Para identificar se uma transação ocorreu em feriado nacional, o pipeline consulta o endpoint da BrasilAPI (<https://brasilapi.com.br/api/feriados/v1/{ANO}>), que retorna a lista de feriados nacionais do ano solicitado. Uma única chamada por ano cobre todas as transações daquele período. A lista é armazenada em um dicionário em memória durante a execução. O campo `venda_em_feriado` é calculado verificando se a data da transação está contida nessa lista, resultando em um valor booleano (verdadeiro ou falso). Como medida de resiliência, caso a API esteja indisponível, o pipeline utiliza uma lista de feriados nacionais de 2024 embutida no código como fallback, evitando a falha da execução.

3.2.6. Anonimização em conformidade com a LGPD

Dois campos de identificação pessoal recebem tratamento diferenciado antes da carga:

O CPF é pseudonimizado por meio de mascaramento, mantendo visíveis apenas os seis dígitos centrais. A transformação torna a re-identificação impraticável sem acesso à fonte original, atendendo ao princípio de pseudonimização previsto na LGPD:

Entrada: 123.456.789-00
Saída: *.456.789-**

O campo `nome_aluno` é completamente removido do registro final, não apenas mascarado. Nomes próprios são dados que permitem identificação direta (Art. 5º, inciso I da LGPD) e não são necessários para o relatório analítico, portanto sua remoção atende ao princípio de minimização de dados (Art. 6º, inciso III da LGPD).

3.3. Etapa de Carga (Load)

A carga dos dados transformados no banco de dados PostgreSQL é realizada por meio da função `execute_batch()` da biblioteca `psycopg2`, que agrupa múltiplos registros em lotes de 200 para envio ao banco. Esse agrupamento reduz significativamente o overhead de round-trips de rede em comparação ao envio registro a registro, melhorando o desempenho em conjuntos de dados maiores.

A instrução SQL utilizada é um UPSERT implementado com a cláusula `INSERT ... ON CONFLICT ... DO UPDATE SET` do PostgreSQL. Essa abordagem garante a idempotência da operação, conforme detalhado na seção de metodologia de idempotência.

A tabela de destino, `fato_vendas`, foi projetada no modelo estrela simplificado, concentrando em uma única tabela fato todos os atributos relevantes para a análise: dados da transação, dados geográficos enriquecidos, flag de feriado e métricas de engajamento. Três índices foram criados nas colunas com maior probabilidade de uso em filtros analíticos: `data_transacao` (análise temporal), `estado` (análise geográfica) e `venda_em_feriado` (análise de sazonalidade).

3.4. Infraestrutura e Containerização

3.4.1. Idempotência

A estratégia de idempotência adotada utiliza o campo `id_transacao` como chave natural — identificador único atribuído pelo ERP financeiro no momento da transação — declarado como `PRIMARY KEY` na tabela `fato_vendas`. A carga utiliza o comando:

```
INSERT INTO fato_vendas (...) VALUES (...)  
ON CONFLICT (id_transacao) DO UPDATE SET  
    cidade = EXCLUDED.cidade,  
    venda_em_feriado = EXCLUDED.venda_em_feriado, ...
```

O PostgreSQL verifica atomicamente a existência de uma linha com o mesmo `id_transacao`. Se não existir, insere normalmente. Se já existir, atualiza todos os campos com os novos valores. Em nenhum cenário uma transação aparece duplicada na tabela. Essa abordagem foi preferida em relação ao particionamento por data com `overwrite` — que exigiria lógica adicional de gerenciamento de partições — e à tabela

de controle por hash — que requereria uma tabela auxiliar e aumentaria a complexidade operacional.

3.4.2. Resiliência e Tratamento de Falhas

O pipeline implementa três camadas de proteção contra falhas:

a) Retry de tarefas do Airflow: Configurado nos `default_args` do DAG, todas as tarefas possuem política de 2 tentativas com intervalo de 1 minuto entre elas. Se uma tarefa falhar por qualquer motivo (erro de rede, banco temporariamente indisponível), o Airflow aguarda e tenta novamente. Após esgotar todas as tentativas, a tarefa é marcada como failed e o restante do DAG é bloqueado, evitando carga de dados parciais.

a) Backoff exponencial nas chamadas de API: Para as consultas à BrasilAPI, o código implementa até 3 tentativas com espera exponencial entre elas (2 segundos após a primeira falha, 4 segundos após a segunda). Esse padrão evita sobrecarregar o serviço público em momentos de instabilidade temporária.

a) Fallback de dados para feriados: Caso a API de feriados esteja completamente indisponível após todas as tentativas, o pipeline não falha nem descarta os registros. Utiliza uma lista de feriados nacionais de 2024 embutida no código como fallback, registrando em log que o fallback foi acionado para revisão posterior.

4. DIFICULDADES ENCONTRADAS

Durante o desenvolvimento do projeto, foram identificadas dificuldades de ordem técnica e conceitual, documentadas a seguir:

1. Permissões de arquivos no ambiente Docker

O problema mais significativo enfrentado foi relacionado às permissões de sistema de arquivos dentro dos containers Docker. Ao tentar montar os volumes com os arquivos de dados e scripts, o container subia corretamente, porém não conseguia ler os arquivos montados, gerando erros de permissão (Permission Denied). A solução

encontrada foi aplicar a permissão `chmod 500` nos arquivos e configurar o container para executar como `root`. Esse comportamento ocorre porque o usuário padrão dentro da imagem do Airflow possui UID diferente do usuário do sistema operacional `host`, causando conflito nas permissões do sistema de arquivos compartilhado.

2. Inconsistência entre formatos de dados das fontes

As duas fontes de dados apresentaram inconsistências que precisaram ser tratadas na transformação: CPFs em dois formatos distintos (com e sem máscara), datas em formato brasileiro e ISO 8601, e valores monetários com vírgula como separador decimal (padrão brasileiro) em vez de ponto (padrão do PostgreSQL). Cada uma dessas inconsistências exigiu tratamento específico e validação de borda.

3. Configuração da cadeia de dependências do Docker Compose

Garantir que os serviços subissem na ordem correta sem falhas de conexão exigiu entendimento aprofundado do mecanismo de `depends_on` com `conditions` no Docker Compose. A configuração de `healthchecks` para os bancos PostgreSQL e a sincronização com o serviço `airflow-init` (que deve concluir antes do `webserver` e do `scheduler` subirem) demandou iterações até o comportamento correto ser alcançado.

4. Instabilidade da BrasilAPI durante o desenvolvimento

Durante as execuções de teste, a BrasilAPI apresentou períodos de latência elevada e `timeouts` ocasionais, especialmente para o endpoint de CEP. Isso motivou a implementação do mecanismo de cache em arquivo JSON e do `backoff` exponencial nas tentativas de requisição, que se mostraram essenciais para a estabilidade do pipeline.

5. Volumetria do XCom do Airflow

O mecanismo de XCom do Airflow armazena os dados no banco de metadados e possui limitações para grandes volumes de dados. Para o volume de dados do projeto (algumas centenas de registros), o XCom funcionou adequadamente, mas foi necessário ter ciência de que essa abordagem não escalaria para volumes de dados maiores, onde seria necessário adotar armazenamento intermediário em disco ou em um data lake.

5. RESULTADOS

5.1. Resultados Esperados

Com base nos requisitos da atividade, os seguintes resultados eram esperados ao final do desenvolvimento:

- Um DAG funcional no Apache Airflow executando diariamente às 06h00, com grafo de dependências visível na interface web.
- Extração correta dos arquivos CSV (latin-1, delimitador ;) e JSON (UTF-8) sem perda de registros válidos.
- Transformação completa dos dados: padronização de CPF, normalização de datas, junção LEFT JOIN, enriquecimento geográfico via BrasilAPI, detecção de feriados e anonimização conforme LGPD.
- Carga idempotente na tabela fato_vendas do banco PostgreSQL, sem duplicatas após reexecuções.
- Resiliência a falhas de API com tentativas automáticas e fallback de dados.
- Observabilidade completa via painel web do Airflow: grafo, histórico de execuções, logs por tarefa.
- Todo o ecossistema inicializado com um único comando: `docker compose up -d`.

5.2. Resultados Obtidos

Após o desenvolvimento e os testes de execução, os seguintes resultados foram observados:

5.2.1. Pipeline funcional e observável

O DAG nogtech-pipeline foi registrado com sucesso no Airflow e é visível na interface web com seu grafo de dependências completo. Execuções manuais disparadas pela interface concluíram com sucesso, com todas as quatro tarefas (`task_extract_transacoes`, `task_extract_engajamento`, `task_transform` e `task_load`) finalizadas no estado `success`. O histórico de execuções, os logs individuais por tarefa e

os tempos de duração ficaram disponíveis na interface web, atendendo plenamente o requisito de observabilidade.

5.2.2. Dados corretamente transformados e carregados

Os registros do arquivo CSV foram extraídos com o encoding e delimitador corretos, sem perda de caracteres especiais. As transformações de CPF, data, junção LEFT JOIN, enriquecimento geográfico e detecção de feriados foram aplicadas conforme especificado. O campo nome_aluno foi removido do conjunto de dados e o CPF foi mascarado antes da carga. Os dados foram gravados com sucesso na tabela fato_vendas do banco postgres-nogtech, verificado por consulta direta ao banco via cliente PostgreSQL.

5.2.3. Idempotência verificada

A execução do pipeline múltiplas vezes com os mesmos arquivos de dados de entrada resultou no mesmo conjunto de registros na tabela fato_vendas, sem duplicatas. A operação UPSERT via ON CONFLICT funcionou conforme esperado, atualizando os campos quando o id_transacao já existia na tabela.

5.2.4. Cache e resiliência funcionando corretamente

O arquivo cep_cache.json foi criado após a primeira execução e utilizado nas execuções seguintes, reduzindo as chamadas à BrasilAPI para os CEPs já consultados. Em testes com a API de feriados simulando indisponibilidade, o fallback foi acionado corretamente e registrado em log, sem interromper a execução do pipeline.

5.2.5. Pontos de melhoria identificados

Durante o desenvolvimento foram identificados dois pontos de melhoria que não comprometem os requisitos da atividade, mas seriam relevantes em um ambiente de produção:

- O volume de cache de CEPs não está mapeado no Docker Compose, o que significa que o cache não persiste entre recriações do container. Em produção,

seria necessário adicionar o mapeamento `./cache:/opt/airflow/cache` ao `compose.yml`.

- O fallback de feriados cobre apenas o ano de 2024. Para dados de outros anos, o pipeline confia inteiramente na disponibilidade da API. Uma cobertura mais ampla de fallback aumentaria a resiliência.

6. CONCLUSÃO

O desenvolvimento deste projeto proporcionou uma experiência prática abrangente nos principais conceitos da Engenharia de Dados moderna. A implementação de um pipeline de ETL completo — desde a extração de arquivos heterogêneos até a carga em um Data Warehouse relacional, passando por transformações de negócio e enriquecimento via API pública — evidenciou a complexidade real que esses sistemas enfrentam em ambientes de produção.

A escolha do Apache Airflow como ferramenta de orquestração mostrou-se acertada para o cenário proposto. A capacidade de expressar o pipeline inteiramente em Python, com versionamento nativo em Git, combinada com a interface web completa para monitoramento e a facilidade de agendamento por expressões cron, ofereceu a melhor relação custo-benefício entre as opções disponíveis. O paradigma de DAGs do Airflow, com dependências explícitas entre tarefas e execuções paralelas quando possível, reflete diretamente a estrutura lógica do problema.

Do ponto de vista técnico, os conceitos de idempotência — garantidos pelo UPSERT via chave natural — e de resiliência — implementada com tentativas automáticas, backoff exponencial e fallback de dados — revelaram-se indispensáveis para a construção de sistemas de dados confiáveis. Um pipeline que falha silenciosamente ou que duplica dados em reexecuções é inaceitável em ambientes analíticos onde as decisões de negócio dependem da acurácia dos dados.

A conformidade com a LGPD, expressa na pseudonimização do CPF e na remoção do nome do aluno, introduziu uma dimensão importante ao projeto: sistemas de dados não existem no vácuo regulatório. A privacidade dos dados pessoais deve ser

considerada desde a concepção do pipeline (privacy by design), e não como uma camada adicionada ao final.

A containerização via Docker Compose consolidou o entendimento de que a reprodutibilidade do ambiente é tão importante quanto o código em si. A premissa de que o pipeline deve funcionar igualmente em qualquer máquina com Docker instalado — sem dependências do sistema operacional local — é um requisito fundamental para equipes de dados que trabalham de forma colaborativa.

Em síntese, o projeto cumpriu todos os requisitos funcionais e de engenharia estabelecidos no enunciado da atividade, entregando um pipeline de ETL funcional, resiliente, idempotente, observável e em conformidade com a legislação de proteção de dados. As dificuldades encontradas, em especial as relacionadas às permissões de arquivos no Docker e à instabilidade da API externa, contribuíram para o aprofundamento do entendimento sobre os desafios reais da operação de sistemas de dados em produção.